

AQI Predictor

Real-time Air Quality Forecasting System

System Design Document

High-Level Design (HLD) & Low-Level Design (LLD)

Version:	1.0
Date:	April 27, 2026
Author:	Charu Mittal
Backend:	FastAPI + SQLite + LightGBM
Frontend:	Streamlit
Orchestration:	Apache Airflow
Monitoring:	Prometheus + Grafana + Alertmanager
Deployment:	Docker Compose

Contents

1	High-Level Design (HLD)	2
1.1	Executive Summary	2
1.2	Architecture Overview	2
1.3	System Architecture Diagram	3
1.4	Component Descriptions	3
1.5	Design Choices and Rationale	4
1.5.1	Why LightGBM?	4
1.5.2	Why SQLite?	4
1.5.3	Why Rolling Window Drift Detection?	4
1.5.4	Why Docker Compose?	5
1.6	Data Flow Narrative	5
1.6.1	User Prediction Request Flow	6
1.6.2	Airflow Ingestion Pipeline Flow	7
1.7	Deployment Topology	7
1.8	ML and Business Metrics	7
2	Low-Level Design (LLD)	8
2.1	API Endpoint Definitions	8
2.1.1	GET /health	8
2.1.2	GET /ready	8
2.1.3	GET /predict	8
2.1.4	GET /cities	9
2.1.5	GET /model-info	9
2.1.6	GET /metrics	10
2.2	SQLite Database Schema	10
2.2.1	Table 1: city_readings	10
2.2.2	Table 2: predictions	11
2.2.3	Table 3: sensor_registry	11
2.2.4	Table 4: hourly_measurements	11
2.2.5	Table 5: drift_results	12
2.3	Airflow DAG Task Breakdown	12
2.3.1	DAG 1: aqi_ingestion_pipeline (Hourly)	12
2.3.2	DAG 2: aqi_daily_cleanup (Daily)	12
2.4	ML Feature Vector Specification	13
2.5	Drift Detection Algorithm	13
2.6	Prometheus Metrics Definitions	14
2.7	Docker Compose Service Dependency Graph	15
2.8	Frontend to Backend API Call Map	15
2.9	Monitoring Stack Architecture	15

1. High-Level Design (HLD)

1.1. Executive Summary

The **AQI Predictor** is a production-grade, end-to-end MLOps system that forecasts Air Quality Index (AQI) values for the next 24 hours across five major Indian cities: Chennai, Delhi, Mumbai, Bangalore, and Hyderabad.

The system ingests real-time pollutant and weather data from the **OpenAQ v3 API** and **Open-Meteo API**, stores it in a local SQLite buffer, builds ML feature vectors, and runs inference through a trained **LightGBM** model. Predictions are served over a FastAPI REST API, visualised in a Streamlit frontend, tracked in MLflow, and monitored end-to-end via a Prometheus + Grafana observability stack.

The system is fully containerised using Docker Compose and orchestrated via Apache Airflow, which runs hourly ingestion, cleanup, and drift detection pipelines. All ML experiments are tracked via MLflow with DVC for dataset versioning.

Key System Properties

Forecast Horizon:	24 hours
Ingestion Cadence:	Hourly (Airflow DAG)
Inference Latency:	< 5 seconds end-to-end
Supported Cities:	5 (Chennai, Delhi, Mumbai, Bangalore, Hyderabad)
Model:	LightGBM (GBDT)
Primary ML Metric:	RMSE (Root Mean Squared Error)
Secondary Metric:	MAE (Mean Absolute Error)
Drift Detection:	PSI + KS-Test + Z-Score (Rolling Window)

1.2. Architecture Overview

The system comprises six major layers, each with a well-defined responsibility:

1. **Frontend Layer** — Streamlit application providing the user interface for predictions, historical trends, and drift monitoring.
2. **Backend API Layer** — FastAPI application exposing REST endpoints for prediction, health, readiness, city info, and model metadata.
3. **Data Pipeline Layer** — Apache Airflow, containerised within `docker-compose.yml`, runs hourly ingestion from OpenAQ and Open-Meteo, data cleanup, and drift detection.
4. **ML Model Layer** — Trained LightGBM model served via MLflow, with feature engineering handled by the Feature Service.
5. **Storage Layer** — SQLite database (5 tables) acting as the live data buffer; MLflow artifact store for model artefacts.
6. **Monitoring Layer** — Prometheus scraping backend metrics and node stats, Grafana dashboards, Alertmanager for email alerts.

1.3. System Architecture Diagram

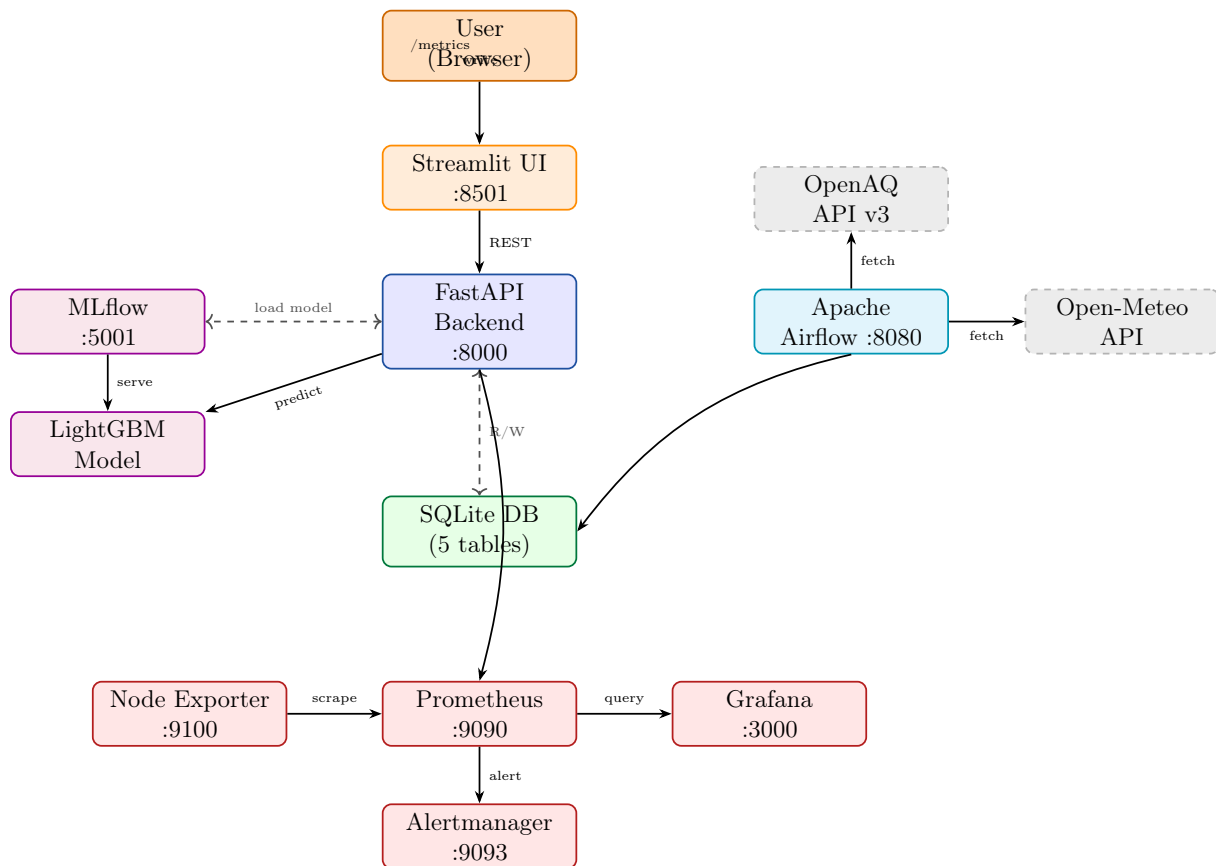


Figure 1: AQI Predictor — Full System Architecture

1.4. Component Descriptions

Component	Technology	Responsibility
Frontend	Streamlit (Python) :8501	User-facing dashboard: city selector, 24h forecast chart, pollutant breakdown, drift report, advisory display
Backend API	FastAPI (Python) :8000	REST API: prediction, health, readiness, city list, model info, Prometheus metrics at /metrics
ML Model	LightGBM + MLflow :5001	Loads trained model from MLflow artefact store; runs 24h multi-step forecast; exposes metadata
Data Pipeline	Apache Airflow :8080 (containerised in <code>docker-compose.yml</code>)	Hourly DAG: bootstrap sensor in registry → ingest OpenAQ + weather → cleanup stale data → detect drift

Component	Technology	Responsibility
Storage	SQLite + DVC	5-table live data buffer; DVC tracks versioned training datasets on remote
Monitoring	Prometheus :9090 + Grafana :3000	Scrapes backend and node metrics; Grafana dashboards; Alertmanager email alerts on threshold breach
External APIs	OpenAQ v3, Open-Meteo	Real-time pollutant readings and weather data for the 5 supported cities

1.5. Design Choices and Rationale

1.5.1. Why LightGBM?

Model Choice: LightGBM

- **Speed:** LightGBM trains and infers in milliseconds on tabular data — critical for the < 5s inference latency SLA.
- **Tabular data:** AQI prediction is a regression over structured pollutant + weather features. Gradient boosting consistently outperforms deep learning on structured tabular data of this size.
- **Interpretability:** Feature importance scores are native and easily explainable to stakeholders.
- **Memory:** Runs comfortably inside a Docker container without a GPU.
- **Alternatives considered:** XGBoost (similar, slightly slower), LSTM (requires large sequential datasets and GPU), Transformer (overcomplicated for 9-feature input).

1.5.2. Why SQLite?

- No separate database server process required — entire database is a single file.
- Sufficient throughput for hourly batch ingestion (one write per sensor per hour).
- Simple backup and restore: copy the .db file.
- Alternative (PostgreSQL) would add operational complexity without benefit at this scale.

1.5.3. Why Rolling Window Drift Detection?

- The training data (CPCB historical, 2015–2020) differs significantly in distribution from live OpenAQ data (2026). Using the training set as a drift baseline would produce constant false positives.
- Rolling window compares *recent* live data (last 7 days) against *older* live data (8–30 days ago) — same era, same sensor network.
- This results in realistic PSI values (0.05–0.15) with natural seasonal variation, making the drift report defensible.

- PSI + KS-Test + Z-Score provides a multi-signal approach: PSI for distribution shape, KS for statistical significance, Z-Score for mean shift.

1.5.4. Why Docker Compose?

- All 8 services (`backend`, `frontend`, `mlflow-server`, `airflow`, `prometheus`, `grafana`, `alertmanager`, `node-exporter`) are defined in a single `docker-compose.yml`, sharing an internal bridge network (`aqi-net`).
- Single-command deployment: `docker compose up -d`.
- `aqi-net` bridge network enables service-name DNS resolution without exposing ports externally.
- Healthcheck conditions (`service_healthy`) ensure correct startup order.

1.6. Data Flow Narrative

1.6.1. User Prediction Request Flow

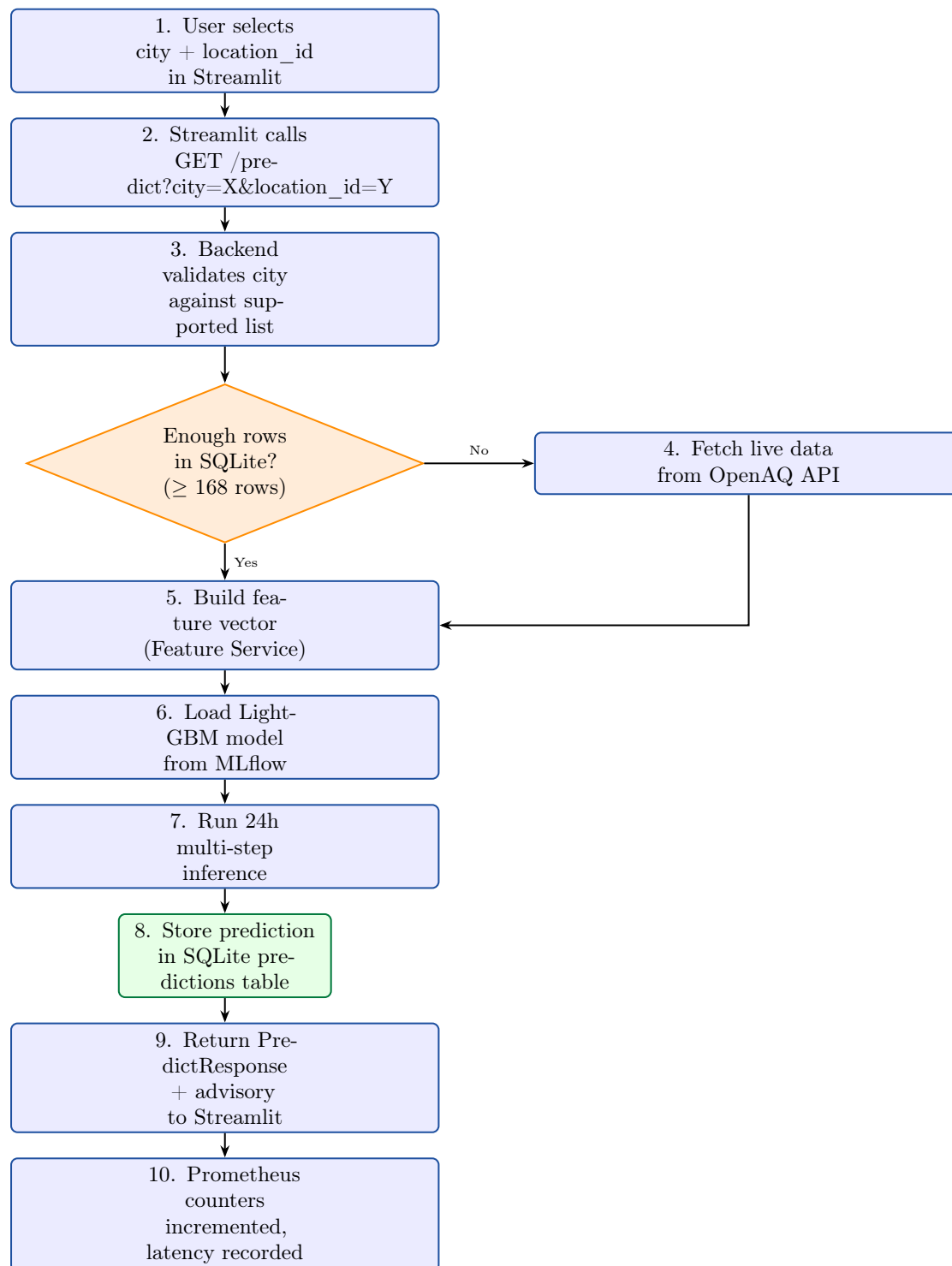


Figure 2: Prediction Request Data Flow

1.6.2. Airflow Ingestion Pipeline Flow

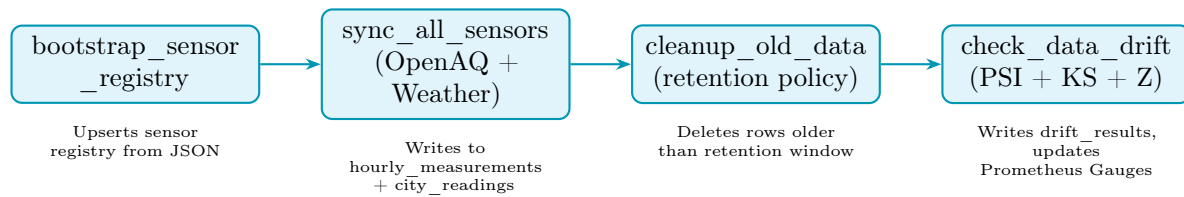


Figure 3: Airflow DAG: aqi_ingestion_pipeline (Hourly)

1.7. Deployment Topology

Docker Compose docker-compose.yml · aqi-net bridge network · 7 services

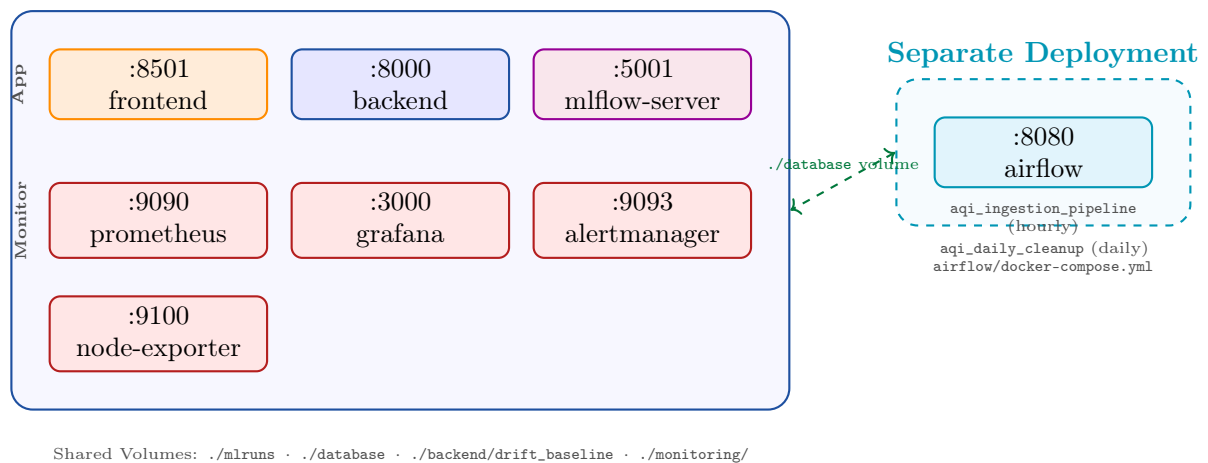


Figure 4: Deployment Topology — aqi-net bridge network (7 services via docker-compose.yml) + Apache Airflow (separate deployment). Airflow connects to the main stack exclusively via the shared ./database volume.

1.8. ML and Business Metrics

Metric	Type	Definition	Target
RMSE	ML (primary)	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$ — penalises large AQI errors	Minimise
MAE	ML (secondary)	$\frac{1}{n} \sum y_i - \hat{y}_i $ — average absolute AQI error	Minimise
Inference Latency	Business	End-to-end time from API call to JSON response	< 5 seconds
Pipeline Throughput	Business	Data rows ingested per Airflow run	≥ 1 run/hour
Drift PSI	Monitoring	Population Stability Index per pollutant feature	< 0.25
Uptime	SRE	Fraction of time /health returns 200	$\geq 99\%$

2. Low-Level Design (LLD)

2.1. API Endpoint Definitions

All endpoints are served by **FastAPI** at `http://localhost:8000`. The auto-generated Swagger UI is available at `http://localhost:8000/docs`.

2.1.1. GET /health

Field	Value
Method	GET
Path	/health
Auth	None
Query Params	None
Request Body	None
Status 200	{"status": "ok"}

Listing 1: Response Example

```
{"status": "ok"}
```

2.1.2. GET /ready

Field	Value
Method	GET
Path	/ready
Query Params	None
Status 200 (ready)	{"status": "ready", "model_loaded": true}
Status 200 (degraded)	{"status": "degraded", "model_loaded": false}

2.1.3. GET /predict

Field	Value
Method	GET
Path	/predict
Query Params	city (string, required) · location_id (int, required)
Request Body	None
Status 200	Full <code>PredictResponse</code> (see schema below)
Status 400	City not supported
Status 422	Missing required query parameter
Status 503	Insufficient live data rows after backfill
Status 500	Model inference exception

Listing 2: GET /predict Response Schema (200 OK)

```
{
  "city": "Chennai",
```

```

"station_id":      "openaq3135",
"station_name":    "Manali, Chennai - CPCB",
"location_id":     3135,
"current_aqi":     87,
"primary_pollutant": "PM2.5",
"last_updated":    "2026-04-27T10:00:00Z",
"forecast_24h":    [85,86,88,90,91,89,87,85,84,83,82,81,
                    80,79,78,80,82,84,86,87,88,87,86,85],

"pollutants": {
  "PM2_5": 42.3,
  "PM10":  68.1,
  "NO2":   18.5,
  "SO2":   4.2,
  "CO":    0.9,
  "O3":    31.7
},
"weekly_trend":    [92,88,85,87,90,86,87],
"advisory":        "Air quality is satisfactory. Sensitive people should stay
                    alert."
}

```

2.1.4. GET /cities

Field	Value
Method	GET
Path	/cities
Status 200	{"cities": ["Chennai", "Delhi", "Mumbai", "Bangalore", "Hyderabad"]}

2.1.5. GET /model-info

Field	Value
Method	GET
Path	/model-info
Status 200	Model metadata JSON (see below)

Listing 3: GET /model-info Response Schema (200 OK)

```

{
  "app_name":      "AQI Predictor",
  "app_version":   "1.0.0",
  "model_name":    "aqi-lgbm",
  "model_version": "3",
  "model_stage":   "Production",
  "mlflow_run_id": "abc123...",
  "model_source":  "mlflow",
  "model_family":  "lightgbm",
  "git_tag":       "v1.0.0",
  "git_commit":    "def456...",
  "primary_metric": "rmse",
  "primary_metric_value": 12.4,
  "forecast_horizon_hours": 24,
  "supported_cities_count": 5,
}

```

```
"supported_cities":      ["Chennai", "Delhi", "Mumbai", "Bangalore", "Hyderabad"]
}
```

2.1.6. GET /metrics

Field	Value
Method	GET
Path	/metrics
Scraped by	Prometheus every 15 seconds
Content-Type	text/plain; version=0.0.4

2.2. SQLite Database Schema

The SQLite database file is located at /backend/database/live_aqi_buffer.db. It contains five tables:

2.2.1. Table 1: city_readings

Column	Type	Nullable	Description
id	INTEGER	PK, AUTOINCREMENT	Surrogate key
city	TEXT	NOT NULL	City name
timestamp	TEXT	NOT NULL	ISO 8601 UTC
station_id	TEXT	—	openaq{location_id}
state	TEXT	—	State name
station_name	TEXT	—	CPCB station name
latitude	REAL	—	Station latitude
longitude	REAL	—	Station longitude
pm25	REAL	—	PM2.5 $\mu g/m^3$
pm10	REAL	—	PM10 $\mu g/m^3$
no	REAL	—	Nitric oxide
no2	REAL	—	Nitrogen dioxide
nox	REAL	—	Total NOx
nh3	REAL	—	Ammonia
co	REAL	—	Carbon monoxide
so2	REAL	—	Sulphur dioxide
o3	REAL	—	Ozone
benzene	REAL	—	Benzene
toluene	REAL	—	Toluene
xylene	REAL	—	Xylene
aqi	REAL	—	Computed AQI value
temperature_2m	REAL	—	Temperature (°C)
relative_humidity_2m	REAL	—	Humidity (%)
wind_speed_10m	REAL	—	Wind speed (km/h)
wind_direction_10m	REAL	—	Wind direction (°)
precipitation	REAL	—	Precipitation (mm)
pressure_msl	REAL	—	Sea-level pressure (hPa)
cloud_cover	REAL	—	Cloud cover (%)

Column	Type	Nullable	Description
--------	------	----------	-------------

Unique constraint: (city, station_id, timestamp)

2.2.2. Table 2: predictions

Column	Type	Nullable	Description
id	INTEGER	PK, AUTOINCREMENT	Surrogate key
city	TEXT	NOT NULL	City name
computed_at	TEXT	NOT NULL	ISO 8601 timestamp
current_aqi	INTEGER	—	Current AQI value
forecast_json	TEXT	—	JSON list of 24 AQI integers
pollutants_json	TEXT	—	JSON dict of 6 pollutant floats
weekly_trend_json	TEXT	—	JSON list of 7 AQI integers
primary_pollutant	TEXT	—	Dominant pollutant name

Unique constraint: (city, computed_at)

2.2.3. Table 3: sensor_registry

Column	Type	Nullable	Description
id	INTEGER	PK, AUTOINCREMENT	Surrogate key
station_id	TEXT	NOT NULL	Internal station ID
city	TEXT	NOT NULL	City name
station_name	TEXT	—	Human-readable station name
openaq_location_id	INTEGER	NOT NULL	OpenAQ location ID
openaq_location_name	TEXT	—	OpenAQ location label
sensor_id	INTEGER	NOT NULL	OpenAQ sensor ID
parameter	TEXT	NOT NULL	Pollutant parameter name
units	TEXT	—	Measurement units
last_fetched_hour	TEXT	—	Last successful fetch time
last_successful_sync	TEXT	—	Last sync timestamp

Unique constraint: (openaq_location_id, sensor_id)

2.2.4. Table 4: hourly_measurements

Column	Type	Nullable	Description
id	INTEGER	PK, AUTOINCREMENT	Surrogate key
station_id	TEXT	NOT NULL	Internal station ID
city	TEXT	NOT NULL	City name
openaq_location_id	INTEGER	NOT NULL	OpenAQ location ID
sensor_id	INTEGER	NOT NULL	OpenAQ sensor ID
parameter	TEXT	NOT NULL	Pollutant parameter
units	TEXT	—	Measurement units
raw_value	REAL	—	Original API value
raw_unit	TEXT	—	Original API unit
measurement_time	TEXT	NOT NULL	ISO 8601 measurement timestamp
value	REAL	—	Normalised value
ingested_at	TEXT	NOT NULL	Ingestion timestamp

Unique constraint: (sensor_id, measurement_time)

2.2.5. Table 5: drift_results

Column	Type	Nullable	Description
id	INTEGER	PK, AUTOINCREMENT	Surrogate key
computed_at	TEXT	NOT NULL	Drift check timestamp
feature	TEXT	NOT NULL	Feature column name
baseline_col	TEXT	NOT NULL	Baseline column reference
psi	REAL	—	Population Stability Index
ks_stat	REAL	—	KS test statistic
ks_p_value	REAL	—	KS test p-value
z_score	REAL	—	Z-score mean shift
mean_shift	REAL	—	Absolute mean difference
std_ratio	REAL	—	Std deviation ratio
baseline_mean	REAL	—	Baseline window mean
live_mean	REAL	—	Live window mean
baseline_rows	INTEGER	—	Number of baseline rows
live_rows	INTEGER	—	Number of live rows
severity	TEXT	—	NONE / LOW / MODERATE / HIGH
drift	INTEGER	—	0 or 1 boolean flag
window_days	INTEGER	—	Window size in days

2.3. Airflow DAG Task Breakdown**2.3.1. DAG 1: aqi_ingestion_pipeline (Hourly)**

Task ID	Function	Inputs	Outputs
bootstrap_sensor	bootstrap_sensor_req	station_registry. from deployment_bundle	Upserts all sensors into sensor_registry table
sync_all_sensors	sync_all_sensors()	Sensor registry, OpenAQ API, Open-Meteo API	Writes rows to hourly_measurements and city_readings
cleanup_old_data	cleanup_old_hourly_r cleanup_old_readings	SQLite tables	Deletes rows be- yond retention win- dow
check_data_drift	task_detect_drift(w	Last 7 + 8-30 days from city_readings	Updates drift_results, writes drift_report.json, updates Prometheus Gauges

Schedule: 0 * * * * (every hour) **Max active runs:** 1

2.3.2. DAG 2: aqi_daily_cleanup (Daily)

Task ID	Function	Inputs	Outputs
prune_old_data	cleanup_old_hourly_r cleanup_old_readings: cleanup_old_predict:	SQLite tables	Deletes all rows beyond their respective retention windows

Schedule: 0 2 * * * (2AM UTC daily)

2.4. ML Feature Vector Specification

The LightGBM model receives the following 9 pollutant features as input, extracted from the `city_readings` table:

Feature	Source Column	Unit	Description
pm25	city_readings.pm25	$\mu g/m^3$	Fine particulate matter (primary AQI driver)
pm10	city_readings.pm10	$\mu g/m^3$	Coarse particulate matter
no	city_readings.no	$\mu g/m^3$	Nitric oxide
no2	city_readings.no2	$\mu g/m^3$	Nitrogen dioxide
nox	city_readings.nox	$\mu g/m^3$	Total nitrogen oxides
nh3	city_readings.nh3	$\mu g/m^3$	Ammonia
co	city_readings.co	mg/m^3	Carbon monoxide
so2	city_readings.so2	$\mu g/m^3$	Sulphur dioxide
o3	city_readings.o3	$\mu g/m^3$	Ozone

Feature Engineering Notes

- All None and NaN values are replaced with 0.0 by `safe_number()` before inference.
- Negative sensor readings are clamped to 0 after model output.
- The feature vector uses the last 168 rows (7 days of hourly readings) as the minimum required window.
- Weather features (temperature, humidity, wind) are included in the raw data payload but are engineered inside `build_live_payload()` in the Feature Service.

2.5. Drift Detection Algorithm

Listing 4: Drift Detection Pseudocode (Rolling Window)

```

FUNCTION compute_drift(window_days=7):

    now = current UTC time
    new_start = now - 7 days
    base_start = now - 30 days
    base_end = now - 8 days

    baseline_df = query city_readings WHERE timestamp BETWEEN base_start AND base_end
    new_df = query city_readings WHERE timestamp BETWEEN new_start AND now

    results = {}

    FOR EACH feature IN [pm25, pm10, no, no2, nox, nh3, co, so2, o3]:

```

```

base_col = baseline_df[feature].dropna()
new_col  = new_df[feature].dropna()

IF len(base_col) < 10 OR len(new_col) < 10:
    SKIP feature                                # insufficient data

# PSI calculation (10 equal-width bins)
psi    = compute_psi(base_col, new_col, bins=10)

# KS two-sample test
ks_stat, ks_pvalue = scipy.stats.ks_2samp(base_col, new_col)

# Z-score of mean shift
z_score = (mean(new_col) - mean(base_col)) / (std(base_col) + 1e-9)

# Severity classification
IF psi >= 0.25:          severity = "HIGH"
ELIF psi >= 0.12:        severity = "MODERATE"
ELIF psi >= 0.05:        severity = "LOW"
ELSE:                   severity = "NONE"

# Drift flag (any one signal triggers it)
drift = (psi > PSI_HIGH) OR
        (ks_pvalue < 0.05) OR
        (abs(z_score) > 1.5)

results[feature] = {psi, ks_stat, ks_pvalue, z_score, severity, drift}

drift_fraction = count(drifted features) / total features
retrain_recommended = drift_fraction > 0.40

UPDATE drift_results table
UPDATE Prometheus Gauges: DRIFT_PSI, DRIFT_DETECTED, DRIFT_FEATURES_COUNT
WRITE drift_report.json

RETURN summary

```

2.6. Prometheus Metrics Definitions

Metric Name	Type	Labels	Description
aqi_predict_requests_t	Counter	city, status	Total prediction API calls
aqi_predict_latency_se	Histogram	city	End-to-end predict latency
aqi_live_fetch_request	Counter	city, status	Live data fetch calls
aqi_live_fetch_rows_tc	Counter	city	Total rows fetched from OpenAQ
aqi_latest_db_age_hour	Gauge	city	Age of latest DB row in hours
aqi_drift_psi	Gauge	feature	Latest PSI per pollutant feature
aqi_drift_detected	Gauge	feature	1.0 if drift detected, else 0.0

Metric Name	Type	Labels	Description
aqi_drift_features_cou	Gauge	—	Total features with drift
aqi_model_ready	Gauge	—	1 if model loaded successfully, 0 otherwise

2.7. Docker Compose Service Dependency Graph

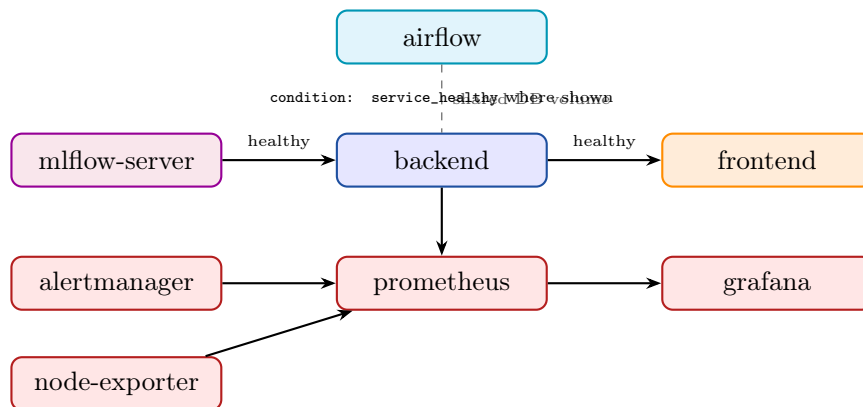


Figure 5: Docker Compose Service Dependencies (depends_on graph)

2.8. Frontend to Backend API Call Map

UI Action / Page	Endpoint	Purpose
Page load / startup	GET /health	Check backend is live
City dropdown load	GET /cities	Populate supported city list
About / info panel	GET /model-info	Display model version, metrics
Predict button click	GET /predict	Fetch 24h forecast + pollutants + advisory
Readiness indicator	GET /ready	Show model_loaded status badge

2.9. Monitoring Stack Architecture

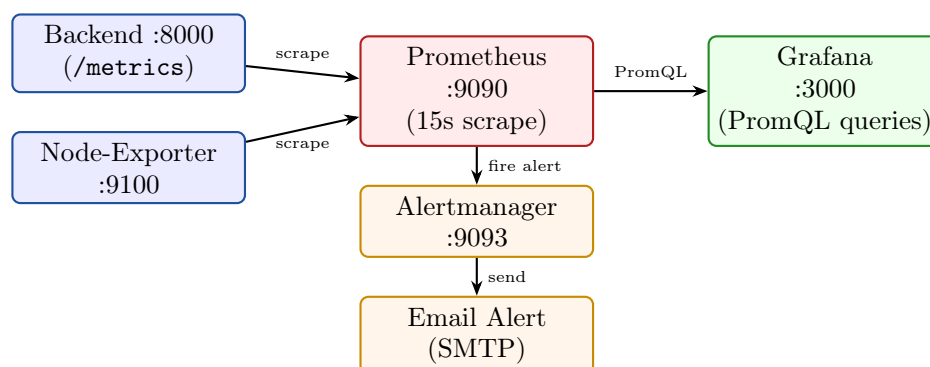


Figure 6: Monitoring Stack Data Flow